Project Goal

Build a production-style pipeline on **GCP + Databricks** that ingests API/CSV/JSON data into **GCS (raw)**, curates it through **Delta Bronze/Silver/Gold**, publishes curated outputs to **BigQuery**, and then (optional) adds orchestration + CI/CD + agentic RAG + HITL.

Working Mode (Agile)

- We work **one step at a time**.
- Each step has:
- **Input** (what you need ready)
- **Do** (exact task)
- **Done when** (clear completion check)
- **Notes** (optional enhancements; we decide to include or defer)

Total Steps (MVP → Production Enhancements)

MVP (core pipeline)

1. **Project naming + repo init**
2. **GCP baseline setup**: APIs enabled, GCS bucket(s), BigQuery datasets
3. **Local dev setup**: Python env, gcloud auth, .env template
4. **Ingestion v1**: pull API or read CSV/JSON → write **raw JSONL** to GCS with run_id
5. **Bronze Delta job**: Databricks reads raw → writes **Delta Bronze** + metadata
6. **Schema contract**: define **Silver target schema** + required fields
7. **Silver transform**: Spark normalize/cast/flatten → write **Delta Silver**
8. **DQ gates + deadletter**: batch checks (null/dup/rowcount/schema) + quarantine table/path
9. **Gold curate**: build training/analytics-ready tables in **Delta Gold**
10. **Publish to BigQuery**: load/overwrite incremental into BQ + basic BQ checks

Production add-ons (optional, time-permitting)

1. **Orchestration**: Databricks Workflows or Composer DAG to run steps 4–10
2. **Ops/Audit**: ops tables for pipeline_runs, dq_results, schema_registry
3. **CI/CD**: GitHub → build/test → deploy jobs/workflows + configs (dev/prod)
4. **Schema drift automation**: detect new schema_hash → route + alert
5. **Agentic RAG MVP**: generate PySpark mapping suggestions
6. **HITL workflow**: PR-based approval for generated code + gated execution
7. **Monitoring**: job metrics, alerts, dashboards

Suggested Folder Layout (simple)

- `src/ingestion/` (API/file → GCS raw)
- `src/dbx/` (Databricks notebooks/scripts: bronze/silver/gold)
- `src/contracts/` (schema definitions, mapping rules)

- `src/dq/` (DQ checks + thresholds)
- `infra/` (later: terraform/scripts)
- `docs/` (architecture notes, decisions)

Decision Log (we'll keep this short)

- **Dataset choice**:
- **Dev/Prod strategy** (separate projects vs datasets):
- **Orchestrator** (Workflows vs Composer):
- **DQ tool style** (Spark checks vs Great Expectations):

Next Step to Execute

Step 1: Project naming + repo init - Input: GitHub account access - Do: create repo + clone + add base folders - Done when: repo exists, initial commit pushed, folder skeleton created